



Introduzione al calcolo ad alte prestazioni (HPC)

Principi della Programmazione Parallela
CdL Informatica - A.A. 2025/2026



Università
di Catania



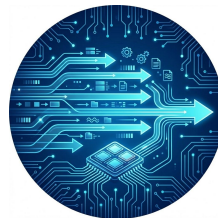
INAF
ISTITUTO NAZIONALE
DI ASTROFISICA

Obiettivi della lezione



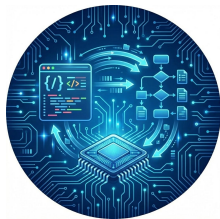
Capire perché esiste l'HPC

Alcuni problemi non sono risolvibili senza calcolo ad alte prestazioni.



Capire perché il calcolo moderno è parallelo

I limiti fisici dell'hardware rendono il parallelismo inevitabile.



Comprendere il legame tra hw, sw e algoritmi

Le prestazioni non sono automatiche ma vanno progettate.



Inquadrare gli obiettivi del corso

Cosa verrà studiato e quali competenze verranno sviluppate.

Perchè esiste l'HPC

Domanda fondamentale

Perché non è sufficiente un computer tradizionale?

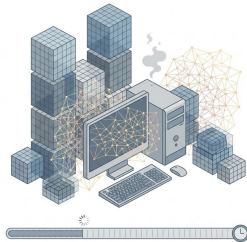
Perchè esiste l'HPC

Domanda fondamentale

Perché non è sufficiente un computer tradizionale?

Limiti del computer singolo

- Problemi **troppo grandi** per un singolo computer
- Problemi **troppo lenti** per tempi accettabili



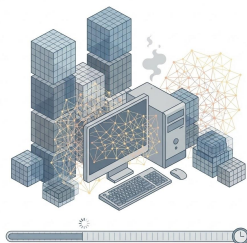
Perchè esiste l'HPC

Domanda fondamentale

Perché non è sufficiente un computer tradizionale?

Limiti del computer singolo

- Problemi **troppo grandi** per un singolo computer
- Problemi **troppo lenti** per tempi accettabili



Vincoli sperimentali

- Problemi troppo costosi o impossibili da testare sperimentalmente



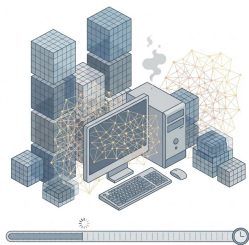
Perchè esiste l'HPC

Domanda fondamentale

Perché non è sufficiente un computer tradizionale?

Limiti del computer singolo

- Problemi **troppo grandi** per un singolo computer
- Problemi **troppo lenti** per tempi accettabili



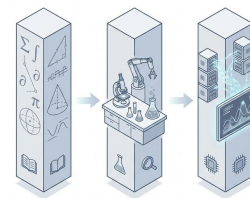
Vincoli sperimentali

- Problemi troppo costosi o impossibili da testare sperimentalmente



Il terzo pilastro

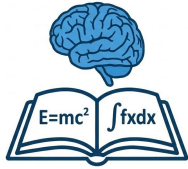
- La **simulazione numerica** come terzo pilastro della scienza



La simulazione come terzo pilastro

Tradizionalmente il metodo scientifico si basa su:

Teoria



- Modelli matematici
- Concetti astratti
- Ipotesi Fondamentali

Esperimento



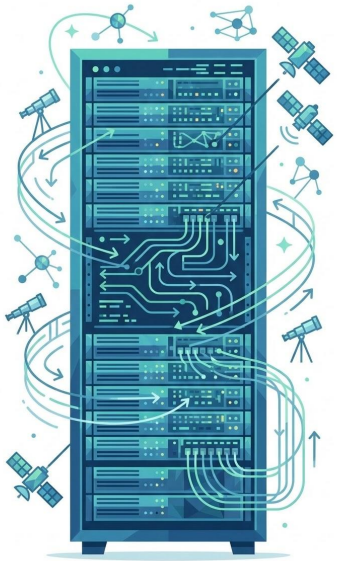
- Osservazioni empiriche
- Raccolta dati
- Validazione

La simulazione come terzo pilastro

Oggi il metodo scientifico si basa su:



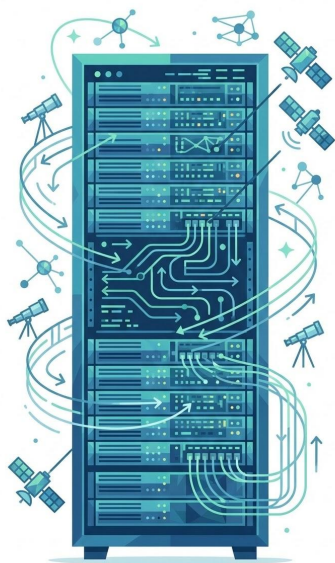
Esempi di problemi HPC



Previsioni meteo e clima

Modelli tridimensionali su larga scala con molte variabili fisiche

Esempi di problemi HPC



Previsioni meteo e clima

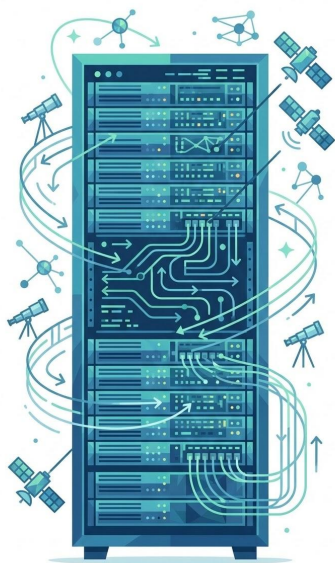
Modelli tridimensionali su larga scala con molte variabili fisiche



Astrofisica e cosmologia

Simulazioni di sistemi complessi non osservabili sperimentalmente

Esempi di problemi HPC



Previsioni meteo e clima

Modelli tridimensionali su larga scala con molte variabili fisiche



Astrofisica e cosmologia

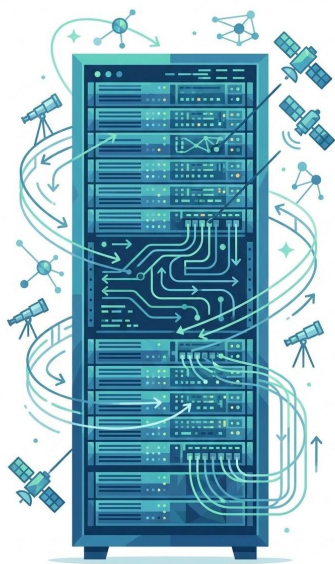
Simulazioni di sistemi complessi non osservabili sperimentalmente



Dinamica molecolare

Calcolo di interazioni tra un numero elevato di particelle

Esempi di problemi HPC



Previsioni meteo e clima

Modelli tridimensionali su larga scala con molte variabili fisiche



Astrofisica e cosmologia

Simulazioni di sistemi complessi non osservabili sperimentalmente



Dinamica molecolare

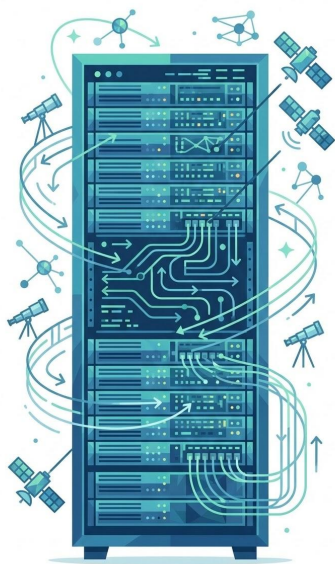
Calcolo di interazioni tra un numero elevato di particelle



Ingegneria computazionale (CFD, strutture)

Analisi numeriche in alternativa o supporto a test sperimentali

Esempi di problemi HPC



Previsioni meteo e clima

Modelli tridimensionali su larga scala con molte variabili fisiche



Astrofisica e cosmologia

Simulazioni di sistemi complessi non osservabili sperimentalmente



Dinamica molecolare

Calcolo di interazioni tra un numero elevato di particelle



Ingegneria computazionale (CFD, strutture)

Analisi numeriche in alternativa o supporto a test sperimentali



Intelligenza artificiale

Addestramento di modelli complessi su grandi quantità di dati

Scalabilità del problema

La sfida della crescita non lineare

- L'aumento del livello di dettaglio comporta una **crescita non lineare** della dimensione del problema
- In problemi tridimensionali, ridurre la dimensione delle celle di un fattore 2 porta a un aumento di circa **8 volte** del numero di elementi
- La quantità di **memoria richiesta** e il numero di **operazioni computazionali** crescono rapidamente

Scalabilità del problema

La sfida della crescita non lineare

- L'aumento del livello di dettaglio comporta una **crescita non lineare** della dimensione del problema
- In problemi tridimensionali, ridurre la dimensione delle celle di un fattore 2 porta a un aumento di circa **8 volte** del numero di elementi
- La quantità di **memoria richiesta** e il numero di **operazioni computazionali** crescono rapidamente

La necessità del parallelismo

- Un approccio di calcolo **puramente sequenziale** non è in grado di gestire questa crescita
- Il **parallelismo** diventa quindi una necessità strutturale, non una semplice ottimizzazione

Che cos'è l'HPC (definizione)

High Performance Computing (HPC) indica l'insieme di tecniche hardware e software utilizzate per risolvere problemi ad **alta intensità computazionale, di memoria o di dati**.

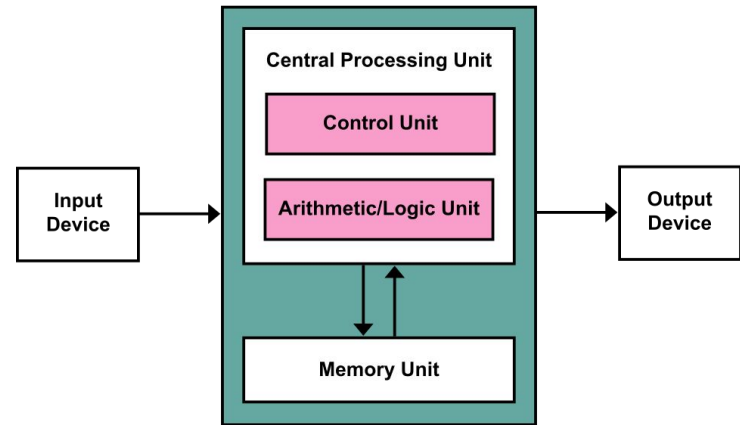
- L'HPC è una **disciplina** che integra:
 - a. architetture di calcolo parallele
 - b. modelli di programmazione
 - c. algoritmi e metodi numerici
 - L'obiettivo dell'HPC è rendere **risolvibili in tempi accettabili** problemi che non scalano su un computer tradizionale
 - Le prestazioni in HPC dipendono dal **corretto bilanciamento** tra hardware, software e algoritmi
 - L'HPC richiede un approccio **esplicitamente parallelo** al calcolo
-

HPC ≠ supercomputer

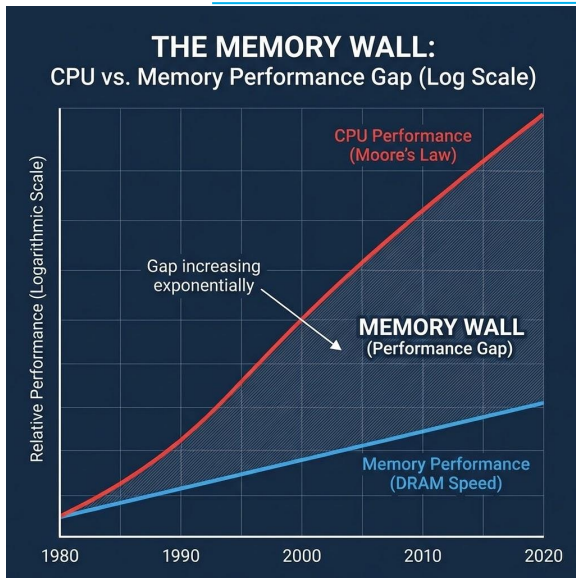
- Il calcolo ad alte prestazioni **non coincide** con l'uso esclusivo di supercomputer di grandi dimensioni.
 - L'HPC riguarda **il modo di utilizzare le risorse**, non solo la loro quantità.
 - Scrivere codice HPC significa progettare software che sfrutti **parallelismo e gerarchia di memoria**, indipendentemente dalla piattaforma.
 - Molte caratteristiche dell'HPC sono presenti anche nei sistemi di uso comune:
 - a. CPU multicore
 - b. unità vettoriali (SIMD)
 - c. acceleratori come GPU
 - Le stesse tecniche HPC si applicano a:
 - a. workstation e laptop moderni
 - b. cluster di piccole e medie dimensioni
 - c. grandi sistemi HPC
-

Il modello sequenziale classico

- Il modello di calcolo sequenziale classico si basa sull'**architettura di Von Neumann**
 - Le principali assunzioni del modello sono:
 - a. esecuzione di **una istruzione alla volta**
 - b. **un singolo flusso di controllo**
 - c. accesso a una **memoria concettualmente uniforme**
 - Questo modello è semplice ed efficace per descrivere algoritmi e programmi
 - Tuttavia, rappresenta una **semplificazione astratta** dell'hardware reale
 - I moderni processori implementano numerose forme di parallelismo **non visibili** nel modello sequenziale
-



Perché il modello sequenziale non basta



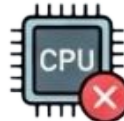
Nei sistemi moderni la **velocità della CPU** è cresciuta molto più rapidamente della **velocità di accesso alla memoria**

- Il tempo di esecuzione di un programma è spesso dominato dall'**attesa dei dati**, non dal calcolo
- Il modello sequenziale assume un accesso alla memoria **uniforme e immediato**, che non riflette la realtà

Le cause dello spreco di tempo di clock



latenze di memoria



cache miss



dipendenze tra istruzioni

La conseguenza

Scrivere codice sequenziale corretto **non garantisce** buone prestazioni sui sistemi moderni

Moore e Dennard

L'era d'oro dello scaling



- **Legge di Moore:** il numero di transistor raddoppia ogni 18–24 mesi
- **Scaling di Dennard:** frequenza può aumentare mantenendo **potenza e dissipazione costanti**
- **Risultato:** aumento automatico delle prestazioni, esecuzione più veloce del codice sequenziale

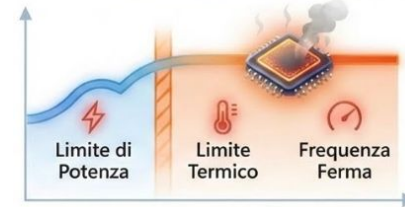
Moore e Dennard

L'era d'oro dello scaling



- **Legge di Moore:** il numero di transistor raddoppia ogni 18-24 mesi
- **Scaling di Dennard:** frequenza può aumentare mantenendo **potenza e dissipazione costanti**
- **Risultato:** aumento automatico delle prestazioni, esecuzione più veloce del codice sequenziale

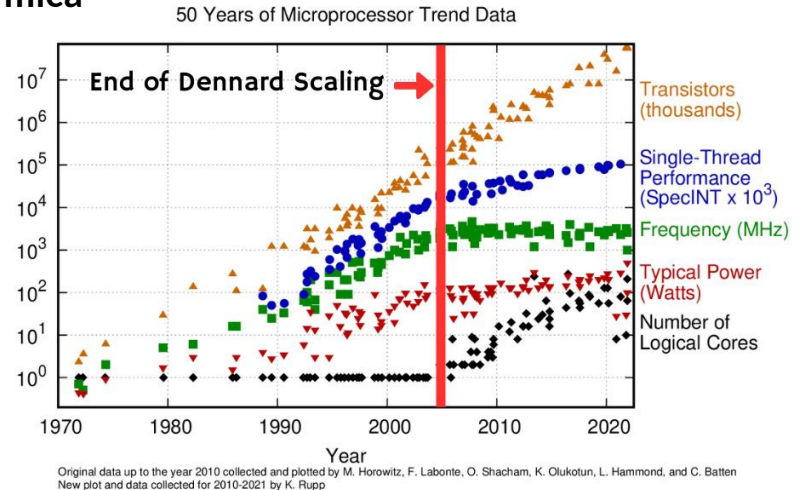
La fine dello scaling di Dennard



- Intorno al 2006 lo **scaling di Dennard si è arrestato**
- **Conseguenza:** l'aumento delle prestazioni **non è più garantito** dall'hardware
- **Software responsabile:** codice sequenziale = hardware sprecato

La fine della free lunch

- L'arresto dello scaling di Dennard ha impedito ulteriori aumenti significativi della **frequenza di clock**
- A partire dalla metà degli anni 2000:
 - a. le frequenze sono quasi ferme
 - b. i limiti principali sono **potenza e dissipazione termica**
- Il codice sequenziale **non diventa più veloce da solo**
- Le prestazioni devono essere ottenute tramite:
 - a. parallelismo
 - b. migliore utilizzo delle risorse
 - c. progettazione consapevole del software



Conseguenza per il software



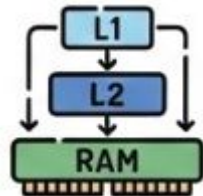
Le prestazioni del software **non migliorano più automaticamente** con l'evoluzione dell'hardware



Il software deve sfruttare esplicitamente il **parallelismo**



Scrivere codice **corretto** non è più sufficiente per ottenere buone prestazioni



Utilizzare in modo efficiente la **gerarchia di memoria**

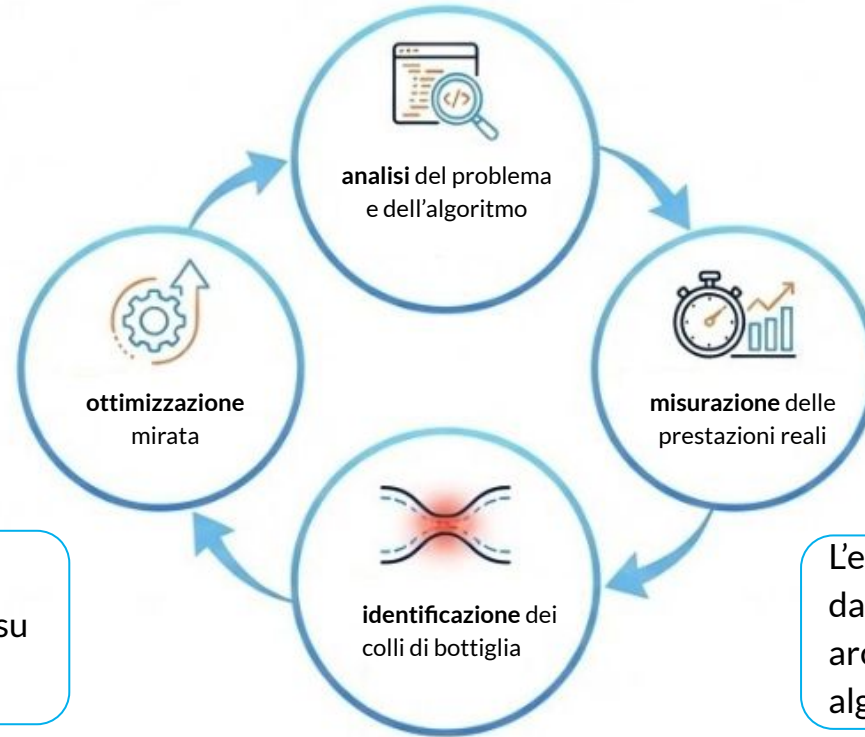


Le prestazioni diventano una **responsabilità del programmatore**

È necessario adottare un approccio di **performance engineering**:
analisi, misurazione e ottimizzazione

Performance engineering

Il **performance engineering** è un approccio sistematico alla progettazione di software efficiente



Le decisioni devono essere basate su **dati misurati**, non su intuizione

L'efficienza dipende dall'interazione tra: architettura, software e algoritmi

La risposta dell'hardware

- L'impossibilità di aumentare ulteriormente la frequenza ha portato a un **cambio di strategia** nella progettazione dell'hardware.
 - L'aumento delle prestazioni è stato ottenuto introducendo **forme di parallelismo** a diversi livelli:
 - a. **Multicore**: più core sullo stesso chip
 - b. **SMT / Hyper-Threading**: più thread hardware per core
 - c. **Unità vettoriali**: una istruzione su più dati
 - d. **Acceleratori** come GPU
 - L'hardware moderno è quindi **intrinsecamente parallelo**.
 - Per sfruttare queste architetture è necessario **software esplicitamente parallelo**.
-

Parallelismo ovunque

- Il parallelismo nei sistemi moderni può essere classificato su **più livelli gerarchici**.
 - Principali livelli di parallelismo:
 - a. **Bit-level parallelism**: operazioni su parole di dimensione crescente;
 - b. **Instruction-level parallelism (ILP)**: pipeline ed esecuzione fuori ordine;
 - c. **Data-level parallelism**: operazioni vettoriali (SIMD);
 - d. **Thread-level parallelism**: esecuzione concorrente di più thread;
 - e. **Process-level parallelism**: esecuzione distribuita su più processi o nodi;
 - Ogni livello contribuisce alle prestazioni complessive.
 - Ignorare uno o più livelli porta a un **uso non efficiente dell'hardware**.
-

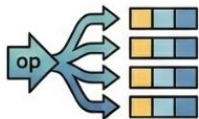
Classificazione del parallelismo

Dal punto di vista del software, il parallelismo può essere classificato in base a **come il lavoro viene suddiviso**

Classificazione del parallelismo

Dal punto di vista del software, il parallelismo può essere classificato in base a **come il lavoro viene suddiviso**

Data Parallelism

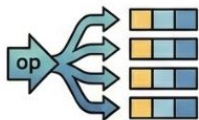


- la stessa operazione viene applicata a **dati diversi**
 - tipico di simulazioni numeriche e array multidimensionali
-

Classificazione del parallelismo

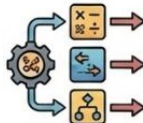
Dal punto di vista del software, il parallelismo può essere classificato in base a **come il lavoro viene suddiviso**

Data Parallelism



- la stessa operazione viene applicata a **dati diversi**
- tipico di simulazioni numeriche e array multidimensionali

Task Parallelism

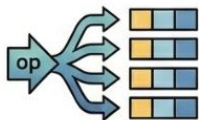


- compiti diversi vengono eseguiti in parallelo
- tipico di workflow complessi o applicazioni modulari

Classificazione del parallelismo

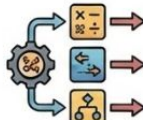
Dal punto di vista del software, il parallelismo può essere classificato in base a **come il lavoro viene suddiviso**

Data Parallelism



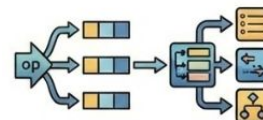
- la stessa operazione viene applicata a **dati diversi**
- tipico di simulazioni numeriche e array multidimensionali

Task Parallelism



- compiti diversi vengono eseguiti in parallelo
- tipico di workflow complessi o applicazioni modulari

Modelli ibridi



- combinazione di data e task parallelism
- comuni nei codici HPC reali

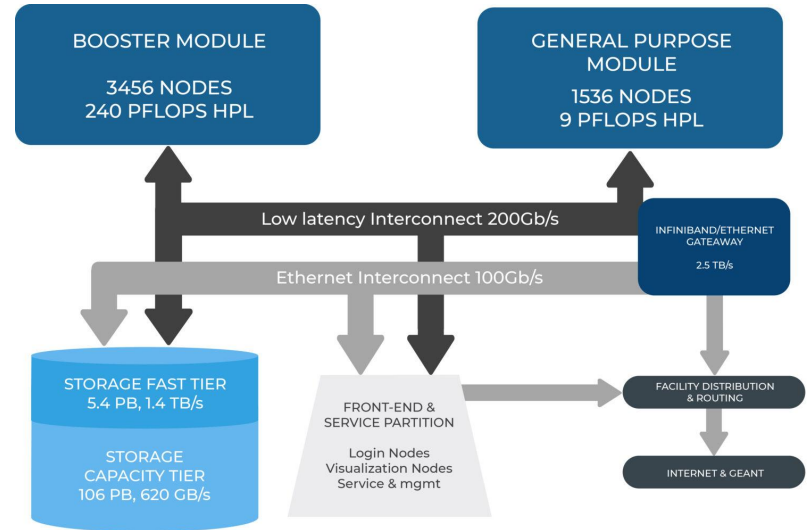
La scelta del tipo di parallelismo dipende: dalla struttura del problema e dall'architettura di esecuzione

HPC = Parallel Computing

- Il calcolo ad alte prestazioni si basa sull'uso **sistematico del parallelismo**
 - L'HPC richiede:
 - a. **pensare** i problemi in termini paralleli
 - b. **programmare** in modo esplicitamente parallelo
 - c. **misurare** e analizzare le prestazioni
 - Il parallelismo non è un dettaglio implementativo, ma una **proprietà strutturale** del calcolo moderno
 - Ignorare il parallelismo significa **non sfruttare l'hardware disponibile**
 - L'HPC integra:
 - a. architettura
 - b. software
 - c. algoritmi
-

Architettura tipica HPC

- Un sistema HPC è tipicamente organizzato come un **cluster di nodi di calcolo**.
- I nodi possono includere **acceleratori**, come **GPU**, per specifiche classi di calcolo.
- I nodi sono collegati tramite una **rete ad alte prestazioni**:
 - a. bassa latenza;
 - b. alta banda;
- I dati sono gestiti tramite **storage condiviso**, accessibile da tutti i nodi.
- Il sistema è progettato per supportare **esecuzione parallela e scalabile**.



Architettura sistema Leonardo (Cineca)

In questo video, prodotto da CINECA viene presentata l'architettura di Leonardo
<https://youtu.be/CFSOY3rWwo4>

Cos'è un cluster

- Un **cluster** è un insieme di **nodi di calcolo indipendenti**, collegati tramite una rete ad alte prestazioni
 - Ogni nodo dispone di:
 - a. proprie CPU, memoria e acceleratori;
 - b. un sistema operativo completo;
 - I nodi **non condividono la memoria principale**: la memoria è fisicamente distribuita.
 - Il cluster viene visto dall'utente come una **singola risorsa computazionale**.
 - Le applicazioni HPC devono gestire in modo esplicito:
 - a. la distribuzione dei dati;
 - b. la comunicazione tra nodi;
 - La comunicazione avviene tramite **scambio di messaggi** sulla rete.
-

HPC software stack

- Un sistema HPC è gestito tramite uno **stack software stratificato**
 - I principali livelli dello stack sono:
 - a. **Applicazioni:** codici scientifici e ingegneristici
 - b. **Librerie:** supporto al parallelismo e al calcolo numerico (es. MPI, OpenMP)
 - c. **Runtime:** gestiscono l'esecuzione parallela
 - d. **Scheduler:** allocano le risorse e gestiscono i job
 - e. **Compilatori:** traducono il codice e ottimizzano per l'architettura
 - Ogni livello svolge un ruolo specifico nel garantire:
 - a. correttezza
 - b. prestazioni
 - c. scalabilità
 - L'efficienza complessiva dipende dall'**interazione tra tutti i livelli**
-

Scheduler

Lo **scheduler** è il componente del sistema HPC che **gestisce e assegna le risorse di calcolo**

Le risorse gestite
includono:

- nodi di calcolo
- CPU e core
- GPU
- memoria
- tempo di esecuzione



Gli utenti non eseguono i programmi direttamente, ma **sottomettono job**

Lo scheduler decide:

- **quando** un job viene eseguito
- **dove** viene eseguito
- **con quante risorse**

L'obiettivo dello scheduler è garantire:

- uso efficiente delle risorse
- equità tra gli utenti
- rispetto delle politiche del sistema

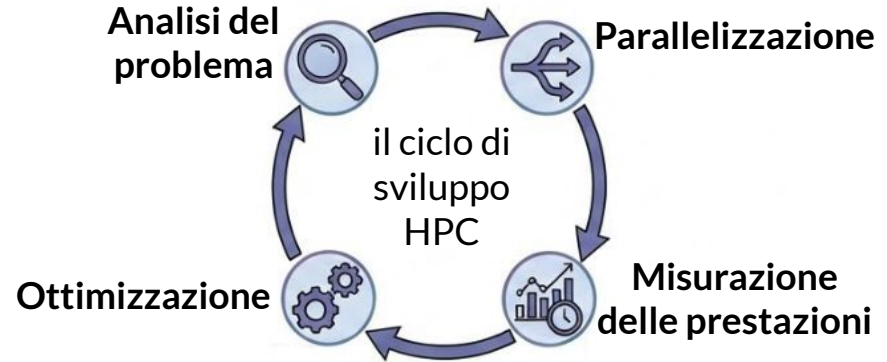
Programmare HPC

Progettare il parallelismo

Programmare HPC significa **progettare software per sistemi paralleli**. Non basta scrivere codice corretto: è necessario **pensare il problema in termini paralleli** e gestire esplicitamente **dati, comunicazione e sincronizzazione**

Requisiti fondamentali

Richiede conoscenza dell'**architettura**, uso di **modelli di programmazione parallela** e attenzione alle **prestazioni**



Conclusione



Le prestazioni sono parte integrante della **correttezza del programma**

Errori comuni

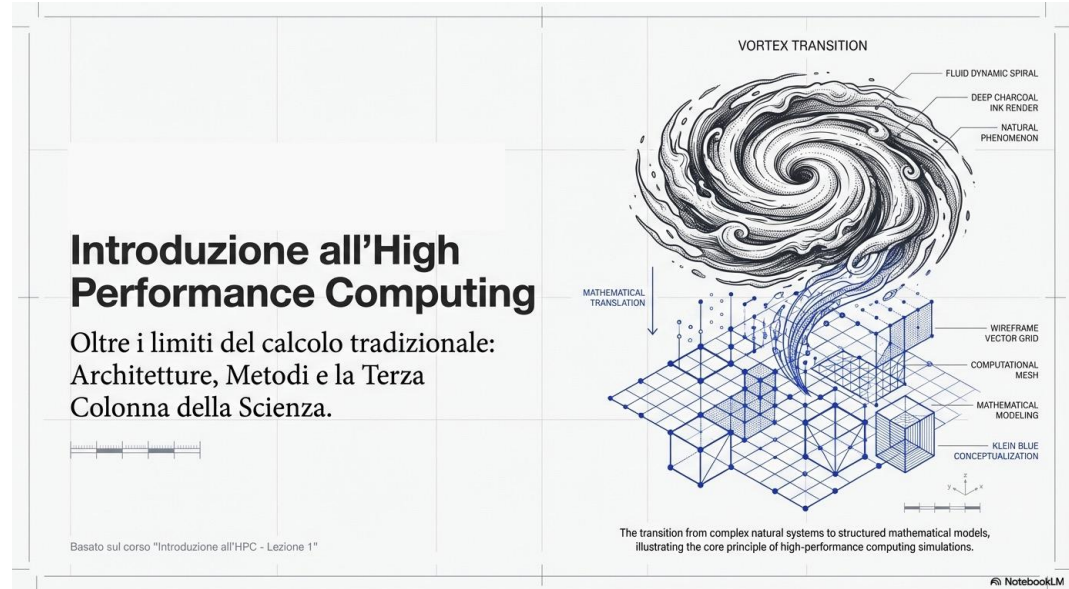
- Pensare che l'HPC significhi solo **avere hardware più potente**
 - Credere che il **compilatore** possa rendere automaticamente efficiente qualunque codice
 - Scrivere codice parallelo senza considerare:
 - a. comunicazione
 - b. sincronizzazione
 - c. località dei dati
 - Ottimizzare senza **misurare** le prestazioni
 - Concentrarsi su micro-ottimizzazioni ignorando:
 - a. struttura dell'algoritmo
 - b. scalabilità del problema
-

Podcast

Questo podcast è stato generato con strumenti di Intelligenza Artificiale (<https://notebooklm.google/>) utilizzando contenuti e fonti forniti dall'autore del corso.

Il materiale ha esclusivamente funzione di supporto e sintesi degli argomenti trattati a lezione. Non sostituisce il materiale didattico ufficiale, le slide, i testi di riferimento o lo studio individuale.

Il podcast deve essere inteso come uno strumento integrativo per il ripasso e il consolidamento dei concetti, e non come fonte esaustiva o alternativa allo studio approfondito.



<https://youtu.be/PzGW0j5Bgmk>